
Casos de Uso

Dos requisitos à implementação



Prof. Dr. João Choma

Universidade Estadual de Maringá — DIN/CTC

github.com/JoaoChoma

Análise de Sistemas de Software ■ Requisitos e Scrum



Questão norteadora

Como transformar necessidades do sistema em interações, funcionalidades e trabalho de desenvolvimento?



Até aqui, já discutimos:

DOMÍNIO DO PROBLEMA



MAPA DE USUÁRIOS

MAPA DE FLUXOS

MAPA DE DADOS



REGRAS DE NEGÓCIO



REQUISITOS

```
+-----+
| Funcionais   |
| Não Funcionais|
+-----+
```



?

A pergunta agora é:

Como representar, de maneira organizada, como os usuários utilizarão as funcionalidades do sistema?



O problema dos requisitos isolados

Considere alguns requisitos:

RF01 - O sistema deve permitir cadastrar clientes.

RF02 - O sistema deve permitir registrar pedidos.

RF03 - O sistema deve calcular o valor total do pedido.

RF04 - O sistema deve permitir cancelar pedidos.

Eles dizem **o que o sistema deve fazer**.

Mas ainda existem perguntas:

- Quem cadastra clientes?
- Quem registra pedidos?
- O que precisa acontecer antes?
- Qual é a sequência de interação?
- O que acontece se algo der errado?
- Quais regras precisam ser respeitadas?



Um requisito funcional pode ser simples:

RF02 — O sistema deve permitir registrar pedidos.

Mas implementar isso exige compreender uma interação:

```
Usuário  
  ↓  
inicia registro  
  ↓  
informa cliente  
  ↓  
adiciona itens  
  ↓  
sistema calcula valores  
  ↓  
usuário confirma  
  ↓  
sistema registra pedido
```

Essa interação representa um **caso de uso**.

Por que os Casos de Uso existem?



Casos de uso ajudam a diminuir a distância entre:

O QUE O SISTEMA DEVE FAZER



REQUISITOS



COMO O USUÁRIO INTERAGE COM O SISTEMA



CASOS DE USO



O QUE PRECISA SER IMPLEMENTADO

Eles ajudam a transformar requisitos abstratos em **cenários de utilização compreensíveis**.



Até agora, nossa pergunta principal era:

O que o sistema precisa fazer?

Com casos de uso, passamos também a perguntar:

Quem utiliza essa funcionalidade?

Qual objetivo essa pessoa deseja alcançar?

Como o sistema e o usuário interagem para alcançar esse objetivo?



O que é um Caso de Uso?

Um **Caso de Uso** descreve uma interação entre um **ator** e o **sistema** para alcançar um **objetivo específico**.

Estrutura conceitual:

```
ATOR
↓
possui um objetivo
↓
interage com
↓
SISTEMA
↓
produz um resultado
```

Exemplo:

```
VENDEDOR
↓
Registrar Pedido
↓
SISTEMA DE VENDAS
↓
Pedido registrado
```



Caso de Uso não é apenas um desenho

Um erro comum:

"Caso de uso é aquele diagrama com bonequinhos."

Não.

O diagrama é apenas uma **visão resumida**.

Um caso de uso pode envolver:

- 1 objetivo;
- 2 atores;
- 3 condições para execução;
- 4 fluxo principal;
- 5 fluxos alternativos;
- 6 exceções;
- 7 regras de negócio;
- 8 requisitos relacionados;
- 9 resultado esperado.



Caso de Uso textual

Explica **como** a interação acontece.

UC03 - Registrar Pedido

Ator: Vendedor

1. Vendedor inicia um novo pedido.
2. Sistema solicita o cliente.
3. Vendedor seleciona o cliente.
4. Vendedor adiciona produtos.
5. Sistema calcula o total.
6. Vendedor confirma.
7. Sistema registra o pedido.

Diagrama de Casos de Uso

Mostra **quem** interage com quais funcionalidades.



Antes de avançar: o que é um ator?

Um **ator** representa um papel externo que interage com o sistema.

Pode ser:

- uma pessoa;
- outro sistema;
- um equipamento;
- um serviço externo;
- uma organização.

Exemplos:

```
Cliente  
Vendedor  
Administrador  
Gateway de Pagamento  
Sistema Bancário  
Serviço de E-mail
```



O ator representa um **papel**.

Tiago

normalmente não seria um bom ator.

Melhor:

Professor
Coordenador
Administrador
Cliente
Vendedor

Uma mesma pessoa pode assumir diferentes papéis.



Como identificar atores?

Pergunte:

- Quem utiliza o sistema?
- Quem inicia uma operação?
- Quem fornece informações?
- Quem recebe informações?
- Qual sistema externo troca dados conosco?
- Quem precisa atingir algum objetivo usando o sistema?



Pergunte ao ator:

O que você precisa conseguir fazer utilizando este sistema?

Exemplo:

ATOR: Vendedor

O que precisa fazer?→

- Cadastrar Cliente→
- Consultar Cliente→
- Registrar Pedido→
- Consultar Pedido→
- Cancelar Pedido→
- Emitir Orçamento

Cada objetivo relevante pode indicar um **Caso de Uso**.



Como nomear Casos de Uso?

Uma prática comum é utilizar:

VERBO + OBJETO

Exemplos:

```
Cadastrar Cliente  
Consultar Cliente  
Registrar Pedido  
Cancelar Pedido  
Emitir Nota Fiscal  
Gerar Relatório  
Realizar Pagamento  
Autenticar Usuário
```

O nome deve expressar **uma ação com objetivo identificável**.



Evite modelar diretamente ações de interface como:

```
Clicar no botão salvar  
Abrir tela  
Selecionar combo  
Digitar CPF  
Pressionar ENTER
```

Esses elementos podem aparecer no detalhamento da interface, mas normalmente **não representam objetivos do ator**.

Prefira:

```
Cadastrar Cliente  
Registrar Pedido  
Consultar Estoque
```



Requisitos × Casos de Uso

Os conceitos são relacionados, mas não são iguais.

Requisito	Caso de Uso
Define uma necessidade	Descreve uma interação

Diz o que deve existir Mostra como um objetivo é alcançado

Pode ser funcional ou não funcional Geralmente detalha comportamento funcional

Pode existir isoladamente Normalmente envolve ator + objetivo

Serve como obrigação do sistema Serve como cenário de utilização



Requisito

RF04 - O sistema deve permitir que vendedores registrem pedidos.

Caso de Uso

UC04 - Registrar Pedido

Ator principal: Vendedor

Objetivo:

Registrar uma nova venda para um cliente.

O requisito estabelece a capacidade.

O caso de uso **detalha sua utilização**.



Considere:

RN01 - Todo pedido deve estar associado a um cliente ativo.

RN02 - Produtos sem estoque não podem ser adicionados ao pedido.

RN03 - Descontos acima de 10% exigem autorização do gerente.

Essas regras afetam diretamente o comportamento dos casos de uso.



REGRA DE NEGÓCIO

RN03 - Desconto > 10% exige gerente



REQUISITO

RF07 - Sistema deve controlar autorização de descontos



CASO DE USO

UC04 - Registrar Pedido



FLUXO ALTERNATIVO

Desconto superior a 10%



ATOR SECUNDÁRIO

Gerente

Os artefatos começam a formar uma **cadeia rastreável**.



Rastreabilidade

Um projeto organizado permite responder:

De onde surgiu esta funcionalidade?

Qual regra exige esse comportamento?

Qual caso de uso implementa este requisito?

Qual tarefa implementou esse comportamento?

Exemplo:

```
RN03
↓
RF07
↓
UC04
↓
US12 / Item do Backlog
↓
Sprint 04
↓
TASK-31
```



E os Requisitos Não Funcionais?

Nem todo requisito se transforma diretamente em um caso de uso.

Exemplo:

RNF01 - O sistema deve responder às consultas em até 2 segundos em condições normais de operação.

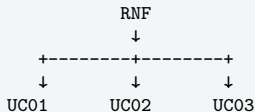
Isso não significa necessariamente criar:

(Responder em 2 segundos)

como caso de uso.



Requisitos não funcionais normalmente estabelecem **qualidades ou restrições** aplicáveis ao sistema ou a partes dele.



Exemplos de categorias:

- desempenho;
- segurança;
- usabilidade;
- disponibilidade;
- confiabilidade;
- compatibilidade;
- manutenibilidade;
- portabilidade.



UC — Realizar Pagamento

Requisitos associados:

RF12 - Permitir pagamento por cartão.

RNF03 - Dados sensíveis devem ser protegidos.

RNF07 - Operação deve responder em até 3 segundos, desconsiderando o tempo do serviço externo.

RNF11 - Falhas de pagamento devem ser registradas.

O caso de uso descreve a interação.

Os RNFs estabelecem **restrições e atributos de qualidade** dessa interação.



Antes do diagrama, precisamos perceber algo importante:

O diagrama não substitui a documentação.

Um Documento de Requisitos pode organizar:

1. Visão e contexto do sistema
2. Stakeholders
3. Regras de negócio
4. Requisitos funcionais
5. Requisitos não funcionais
6. Casos de uso
7. Regras e requisitos relacionados
8. Critérios de aceitação
9. Restrições e premissas



Podemos manter identificadores:

RN01 - Regra de Negócio

RN02 - Regra de Negócio

RF01 - Requisito Funcional

RF02 - Requisito Funcional

RNF01 - Requisito Não Funcional

RNF02 - Requisito Não Funcional

UC01 - Caso de Uso

UC02 - Caso de Uso

Isso facilita a **rastreabilidade**.



Um modelo possível:

UC01 - Nome do Caso de Uso

Objetivo:

Ator principal:

Atores secundários:

Pré-condições:

Pós-condições:

Gatilho:

Fluxo principal:

Fluxos alternativos:

Fluxos de exceção:

Regras de negócio relacionadas:

Requisitos relacionados:



UC04 — Registrar Pedido

Objetivo

Permitir que um vendedor registre um pedido para um cliente.

Ator principal

Vendedor.

Atores secundários

Gerente, quando houver necessidade de autorização de desconto.



Pré-condições

PRE01 - Vendedor deve estar autenticado.

PRE02 - Cliente deve estar cadastrado e ativo.

Pós-condições

POS01 - Pedido deve estar registrado.

POS02 - O estoque deve refletir os itens reservados, quando aplicável.



O que é o Fluxo Principal?

O **fluxo principal** representa o caminho esperado para atingir o objetivo.

Também é chamado de:

happy path

Exemplo:

1. Vendedor solicita novo pedido.
2. Sistema solicita identificação do cliente.
3. Vendedor seleciona o cliente.
4. Sistema apresenta os dados do cliente.
5. Vendedor adiciona produtos.
6. Sistema verifica disponibilidade.
7. Sistema calcula o valor.
8. Vendedor confirma o pedido.
9. Sistema registra o pedido.
10. Sistema informa sucesso.



```
[Vendedor]
|
v
Novo Pedido
|
v
Selecionar Cliente
|
v
Adicionar Produtos
|
v
Verificar Estoque
|
v
Calcular Total
|
v
Confirmar
|
v
[Pedido Registrado]
```



O que acontece se:

- cliente estiver bloqueado?
- produto estiver sem estoque?
- desconto ultrapassar o limite?
- pagamento for recusado?
- conexão cair?
- usuário cancelar?

É aqui que entram os **fluxos alternativos e de exceção**.



Representa um caminho válido diferente do fluxo principal.

Exemplo:

FA01 - Aplicar desconto

No passo 7 do fluxo principal:

1. Vendedor informa um desconto.
2. Sistema calcula o novo valor.
3. Se desconto $\leq 10\%$, sistema aplica o desconto.
4. Fluxo retorna ao passo 8.



FA02 - Desconto superior a 10%

1. Vendedor informa desconto superior a 10%.
2. Sistema identifica necessidade de autorização.
3. Sistema solicita autorização do gerente.
4. Gerente autoriza o desconto.
5. Sistema registra a autorização.
6. Sistema aplica o desconto.
7. Fluxo retorna ao processo de pedido.

Relacionado a:

RN03 - Descontos acima de 10%
exigem autorização do gerente.



Representa uma situação que impede ou altera significativamente a conclusão esperada.

FE01 - Produto sem estoque

1. Sistema identifica estoque insuficiente.
2. Sistema informa indisponibilidade.
3. Produto não é adicionado.
4. Sistema mantém o pedido em edição.

Relacionado a:

RN02 - Produtos sem estoque
não podem ser adicionados ao pedido.



Perceba a evolução:

```
"Registrar Pedido"
  ↓
Selecionar cliente
  ↓
Validar cliente
  ↓
Adicionar produto
  ↓
Consultar estoque
  ↓
Calcular valores
  ↓
Validar desconto
  ↓
Solicitar autorização
  ↓
Registrar pedido
```

Uma frase simples começa a revelar **comportamentos que precisarão ser projetados, implementados e testados.**



Casos de uso também ajudam a pensar:

Como saberemos que essa funcionalidade está correta?

Exemplos:

CA01 - Deve ser possível registrar pedido para cliente ativo.

CA02 - Produto sem estoque não pode ser incluído.

CA03 - Desconto superior a 10% deve exigir autorização.

CA04 - Ao concluir, o sistema deve gerar um identificador único para o pedido.



Observe a conexão:

```
FLUXO PRINCIPAL
↓
CENÁRIO DE TESTE

FLUXO ALTERNATIVO
↓
CENÁRIO DE TESTE

FLUXO DE EXCEÇÃO
↓
CENÁRIO DE TESTE
```

Exemplo:

```
Teste 01 - Registrar pedido válido
Teste 02 - Cliente bloqueado
Teste 03 - Produto sem estoque
Teste 04 - Desconto de 5%
Teste 05 - Desconto de 15% autorizado
Teste 06 - Desconto de 15% recusado
```



Agora podemos falar do Diagrama

Depois de compreender os casos de uso textualmente, podemos construir uma **visão geral das funcionalidades e atores**.

O Diagrama de Casos de Uso permite visualizar:

ATORES

<->

CASOS DE USO

<->

FRONTEIRA DO SISTEMA



Os principais elementos são:

ATOR

0
/|\
/ \

CASO DE USO

(Registrar Pedido)

SISTEMA

```
+-----+
|       |
| ( Registrar Pedido ) |
|       |
+-----+
```



A associação indica que um ator participa de um caso de uso.

Vendedor

0

/|\

/ \

|

|

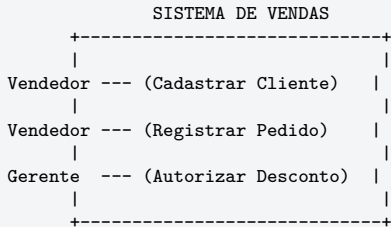
(Registrar Pedido)

Ela responde:

Quem participa dessa funcionalidade?



A fronteira deixa claro o que pertence ao sistema analisado.



Os atores ficam **fora**.

Os casos de uso ficam **dentro**.



flowchart LR

V["Ator Vendedor"]

G["Ator Gerente"]

subgraph S["Sistema de Vendas"]

UC1(["Cadastrar Cliente"])

UC2(["Consultar Cliente"])

UC3(["Registrar Pedido"])

UC4(["Cancelar Pedido"])

UC5(["Autorizar Desconto"])

end

V --- UC1

V --- UC2

V --- UC3

V --- UC4

G --- UC5



Além das associações com atores, podemos encontrar:

```
<<include>>
```

```
<<extend>>
```

generalização

Mas esses relacionamentos devem ser utilizados **quando melhoram a compreensão do modelo**.
Não para deixar o diagrama mais sofisticado.



Utilizado quando um comportamento é **obrigatoriamente reutilizado** por outro caso de uso.

Exemplo:

```
(Registrar Pedido)
|
| <<include>>
v
(Calcular Total)
```

Ao registrar um pedido, calcular o total faz parte obrigatória daquele comportamento modelado.



```
A  
|  
| <<include>>↓
```

B

Pode ser lido como:

*Para executar **A**, o comportamento de **B** é incluído.*

Exemplo:

```
Realizar Compra  
↓  
<<include>>  
↓  
Calcular Frete
```

quando calcular frete for obrigatório naquele processo.



<<extend>>

Representa um comportamento **condicional** ou **opcional** que amplia outro caso de uso em determinada condição.

Exemplo:

```
(Autorizar Desconto)
|
| <<extend>>
v
(Registrar Pedido)
```

Condição:

```
[desconto > 10%]
```

Nem todo pedido precisa dessa autorização.



`<<include>>`

Comportamento incluído
obrigatoriamente

`<<extend>>`

Comportamento adicional
condicionado

Frequentemente reutilizado Ocorre em certas situações

Caso base depende do comportamento Caso base pode ocorrer sem a incluído extensão

Pergunta útil:

Isso sempre acontece?

Pode indicar include.

Isso acontece somente sob determinada condição?

Pode indicar extend.



Não transforme o diagrama em um fluxograma.

Isto é um erro:

```
Login  
↓  
Tela Inicial  
↓  
Selecionar Cliente  
↓  
Adicionar Produto  
↓  
Salvar
```

O Diagrama de Casos de Uso não representa a sequência detalhada de telas.

Ele representa:

atores + objetivos + funcionalidades do sistema.



Criar casos de uso pequenos demais:

(Clicar Salvar)

(Digitar Nome)

(Abrir Tela)

(Selecionar Menu)

Pergunte:

*Isso representa um **objetivo relevante para o ator?***

Se não, provavelmente é apenas uma etapa interna de outro caso de uso.



Uma estratégia prática:

```
REGRAS DE NEGÓCIO
  ↓
REQUISITOS FUNCIONAIS
  ↓
IDENTIFICAR ATORES
  ↓
IDENTIFICAR OBJETIVOS
  ↓
ESPECIFICAR CASOS DE USO
  ↓
IDENTIFICAR RELACIONAMENTOS
  ↓
DIAGRAMA DE CASOS DE USO
```

O diagrama deve ser consequência da análise.

Não o ponto de partida.



MAPAS

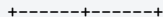
Usuários • Fluxos • Dados



REGRAS DE NEGÓCIO



REQUISITOS



RF



RNF



CASOS DE USO ←---+



ESPECIFICAÇÕES



DIAGRAMA

Mas ainda falta uma pergunta:

Como isso chega ao desenvolvimento?



Em projetos reais, não basta documentar.

Precisamos transformar o conhecimento em **trabalho executável**.

ANÁLISE



FUNCIONALIDADES



BACKLOG



SPRINT



TAREFAS



CÓDIGO



TESTES



INCREMENTO



O Scrum organiza o desenvolvimento em ciclos curtos chamados **Sprints**.

De forma simplificada:

```
PRODUCT BACKLOG
  ↓
SPRINT PLANNING
  ↓
SPRINT BACKLOG
  ↓
SPRINT
  ↓
INCREMENTO
```

A Sprint não é uma fase de análise ou programação.

É um período no qual o time trabalha para atingir um **Sprint Goal** e produzir um incremento útil.



Onde entram os Casos de Uso?

Não existe uma regra do Scrum dizendo:

"Cada caso de uso deve virar uma tarefa."

Na prática, isso seria uma simplificação ruim.

Casos de uso ajudam a compreender funcionalidades.

O backlog organiza **trabalho orientado a valor**.

As tarefas organizam **como o time executará esse trabalho**.



REQUISITOS



CASOS DE USO



FUNCIONALIDADES / CENÁRIOS



ITENS DO PRODUCT BACKLOG



SPRINT BACKLOG



TAREFAS

Mas as relações não precisam ser $1 \rightarrow 1$.

Um caso de uso grande pode originar **vários itens de backlog**.



Caso de Uso

UC04 - Registrar Pedido

Pode ser grande demais para uma única entrega.

Podemos decompor o valor:

Item 1 - Criar pedido para cliente

Item 2 - Adicionar produtos ao pedido

Item 3 - Validar estoque

Item 4 - Aplicar desconto

Item 5 - Autorizar desconto especial

Item 6 - Finalizar pedido



São artefatos diferentes.

Caso de Uso

Descreve detalhadamente a interação.

História de Usuário

Uma forma comum de expressar uma necessidade do backlog:

```
Como <papel>  
quero <funcionalidade>  
para <benefício>.
```

Exemplo:

```
Como vendedor,  
quero adicionar produtos ao pedido  
para registrar os itens solicitados pelo cliente.
```



Não necessariamente.

Podemos utilizar os dois:

CASO DE USO

visão completa da interação



CENÁRIOS / OBJETIVOS



HISTÓRIAS DE USUÁRIO

itens menores orientados a valor



TAREFAS

trabalho técnico

Cada artefato responde a uma pergunta diferente.



Artefato	Pergunta principal
Requisito	O que o sistema precisa atender?
Caso de Uso	Como um ator alcança um objetivo?
História	Qual valor queremos entregar ao usuário?
Tarefa	O que tecnicamente precisa ser feito?

Essa distinção evita misturar **necessidade** com **implementação**.



Requisito

RF04 - Permitir registrar pedidos.

Caso de Uso

UC04 - Registrar Pedido

História

US08 - Como vendedor,
quero adicionar produtos ao pedido
para registrar os itens solicitados pelo cliente.

Tarefas

TASK-21 - Criar endpoint de inclusão de item.
TASK-22 - Implementar validação de estoque.
TASK-23 - Criar interface de inclusão de produto.
TASK-24 - Criar testes automatizados.



Uma abordagem frágil

```
REQUISITO  
↓  
PROGRAMADOR  
↓  
CÓDIGO
```

Problema: muitas decisões ficam implícitas na cabeça de quem implementa.

Uma abordagem mais estruturada

```
REQUISITO  
↓  
CASO DE USO  
↓  
CENÁRIOS  
↓  
CRITÉRIOS DE ACEITAÇÃO  
↓  
BACKLOG  
↓  
TAREFAS  
↓  
IMPLEMENTAÇÃO
```



Sprint Goal

Permitir que vendedores criem pedidos válidos para clientes cadastrados.

Sprint Backlog

US08 - Criar novo pedido
US09 - Adicionar produtos
US10 - Validar estoque
US11 - Finalizar pedido

Esses itens estão relacionados ao:

UC04 - Registrar Pedido



US09 — Adicionar produtos

Como vendedor,
quero adicionar produtos ao pedido
para registrar os itens solicitados.

Possíveis tarefas

TASK-01 - Modelar estrutura de ItemPedido
TASK-02 - Criar endpoint de inclusão
TASK-03 - Implementar regra de quantidade
TASK-04 - Implementar consulta de estoque
TASK-05 - Criar componente de interface
TASK-06 - Implementar testes unitários
TASK-07 - Implementar teste de integração



Elas continuam presentes.

```
RN02
Produto sem estoque não pode ser adicionado.
    ↓
UC04
Registrar Pedido
    ↓
US09
Adicionar Produto
    ↓
CA02
Impedir inclusão sem estoque
    ↓
TASK-04
Implementar validação de estoque
    ↓
TESTE
Estoque = 0 → inclusão recusada
```

Isso é **rastreabilidade ponta a ponta**.



Onde ficam os RNFs?

Também precisam chegar ao desenvolvimento.

Exemplo:

RNF05 - Consulta de produtos deve responder em até 2 segundos.

Pode impactar:

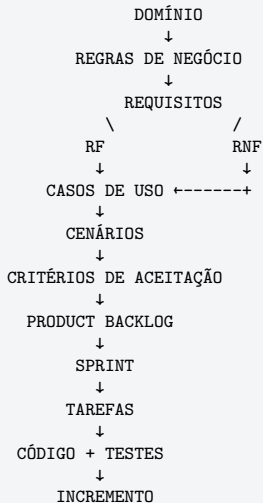
US09 - Adicionar Produto

TASK-08 - Otimizar consulta de produtos

TASK-09 - Criar índice no banco

TASK-10 - Criar teste de desempenho

Requisitos não funcionais não podem desaparecer quando começamos a criar Sprints.





E quando o sistema evolui?

Imagine uma mudança:

RN03 antiga:

Desconto > 10% exige gerente.

RN03 nova:

Desconto > 5% exige gerente.

Com rastreabilidade conseguimos localizar:

RN03

↓

RF07

↓

UC04

↓

FA02

↓

US12

↓

TASK / CÓDIGO / TESTES

A documentação deixa de ser apenas burocracia.

Ela ajuda na **análise de impacto da mudança**.



Um bom caso de uso pode ser discutido por:

CLIENTE / STAKEHOLDER

<->

ANALISTA

<->

PRODUCT OWNER

<->

DESENVOLVEDOR

<->

TESTADOR

Cada pessoa observa o mesmo comportamento por uma perspectiva diferente.



O erro da documentação sem propósito

Documentar tudo não significa documentar bem.

Um artefato deve existir porque ajuda a:

- compreender o domínio;
- reduzir ambiguidades;
- comunicar decisões;
- registrar regras;
- orientar implementação;
- apoiar testes;
- analisar mudanças;
- manter rastreabilidade.

O objetivo não é produzir documentos. \ O objetivo é produzir entendimento compartilhado.



Quanto detalhar?

Depende do projeto.

Um caso de uso crítico pode exigir:

- fluxo principal completo;
- vários fluxos alternativos;
- exceções;
- regras;
- requisitos;
- critérios de aceitação.

Um comportamento simples pode exigir menos documentação.

O nível de detalhe deve ser **suficiente para reduzir ambiguidades relevantes**.



Etapa 1 — Revisar os mapas

Mapa de Usuários

Mapa de Fluxos

Mapa de Dados

Etapa 2 — Revisar

Regras de Negócio

Requisitos Funcionais

Requisitos Não Funcionais



Etapa 3 — Identificar atores

Quem interage com o sistema?

Etapa 4 — Identificar objetivos

O que cada ator precisa alcançar?

Etapa 5 — Criar os casos de uso

UC01
UC02
UC03
...

Etapa 6 — Especificar os casos relevantes

Fluxos, regras, exceções e requisitos.



Etapa 7

Criar o Diagrama de Casos de Uso.

Validar:

- [x] atores corretos?
- [x] objetivos corretos?
- [x] fronteira clara?
- [x] associações corretas?
- [x] include/extend realmente necessários?
- [x] requisitos cobertos?



Etapa 8

Transformar funcionalidades em itens do Product Backlog.

Etapa 9

Definir critérios de aceitação.

Etapa 10

Selecionar itens coerentes com o objetivo da Sprint.

Etapa 11

Decompor o trabalho em tarefas técnicas.



Podemos manter uma tabela:

Regra	Requisito	Caso de Uso	Backlog	Sprint
RN01	RF03	UC01	US02	S01
RN02	RF04	UC04	US09	S02
RN03	RF07	UC04	US12	S03
RN05	RF10	UC06	US15	S03

Isso permite acompanhar o caminho:

NEGÓCIO → SOFTWARE → DESENVOLVIMENTO



Antes de considerar um caso de uso adequado:

- 1 Existe um ator claramente identificado?
- 2 Existe um objetivo relevante?
- 3 O resultado gera valor para o ator?
- 4 As pré-condições estão claras?
- 5 O fluxo principal chega ao objetivo?
- 6 Existem alternativas importantes?
- 7 Existem exceções relevantes?
- 8 As regras de negócio aparecem onde necessário?
- 9 Os requisitos relacionados estão identificados?
- 10 Podemos derivar critérios de aceitação?



O diagrama mostra objetivos ou telas?

Os atores representam papéis?

Os casos de uso possuem nomes claros?

A fronteira do sistema está definida?

Existem casos de uso excessivamente pequenos?

Include e extend possuem justificativa?

O diagrama está compreensível sem explicações enormes?



Considere um sistema acadêmico.

Requisitos:

RF01 - Professor deve registrar notas.

RF02 - Aluno deve consultar notas.

RF03 - Professor deve registrar frequência.

RF04 - Aluno deve consultar frequência.

RF05 - Coordenador deve fechar o período letivo.

Pergunta:

Quais seriam os atores e casos de uso?



Atores

Professor
Aluno
Coordenador

Casos de Uso

Registrar Nota
Consultar Nota
Registrar Frequência
Consultar Frequência
Fechar Período Letivo

Agora podemos começar a investigar **regras, fluxos e exceções**.



Adicione as regras:

RN01 - Nota deve estar entre 0 e 100.

RN02 - Apenas professor responsável pela turma pode alterar notas.

RN03 - Após fechamento do período, notas não podem ser alteradas.

RN04 - Frequência mínima para aprovação é 75%.

Agora pergunte:

Como essas regras modificam nossos casos de uso?



Do modelo estático para o comportamento

Observe nossa evolução no desenvolvimento do sistema:

MAPA DE USUÁRIOS

Quem?

MAPA DE FLUXOS

Como o processo acontece?

MAPA DE DADOS

Quais informações existem?

REGRAS DE NEGÓCIO

Quais regras governam o domínio?

REQUISITOS

O que o software precisa atender?

CASOS DE USO

Como os atores atingem seus objetivos?

BACKLOG

O que entregaremos?

SPRINT

O que entregaremos agora?

TAREFAS

Como construiremos?



Nenhum desses artefatos deve existir isoladamente.

ANÁLISE



MODELAGEM



PLANEJAMENTO



IMPLEMENTAÇÃO



TESTES



ENTREGA



FEEDBACK



O software evolui.

Os modelos também precisam evoluir.



O ponto central

Casos de uso não existem para produzir diagramas.

Eles existem para nos ajudar a responder:

Quem precisa do sistema?

Qual objetivo essa pessoa quer alcançar?

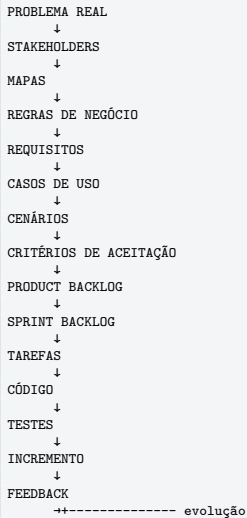
Como o sistema deve se comportar para permitir isso?

Quais regras e restrições precisam ser respeitadas?

Como transformamos esse entendimento em software?



A conexão completa





Agora vocês já possuem elementos para sair de:

"Nosso sistema terá uma funcionalidade de pedidos."

para algo muito mais preciso:

RN → RF/RNF → UC → Cenários



Critérios de Aceitação



Backlog



Sprint



Tarefas



Implementação + Testes



Pegue o projeto da equipe e escolha **uma funcionalidade central**.

Faça o caminho completo:

```
1 Regra de Negócio
  ↓
1 ou mais Requisitos
  ↓
1 Caso de Uso
  ↓
Fluxo Principal
  ↓
Fluxos Alternativos / Exceções
  ↓
Critérios de Aceitação
  ↓
Item(ns) de Backlog
  ↓
Tarefas
```

Se vocês conseguirem manter a rastreabilidade até o final:

*vocês deixaram de apenas descrever uma ideia e começaram a **engenheirar o software**.*



Referências

- BOOCH, G.; RUMBAUGH, J.; JACOBSON, I. *The Unified Modeling Language User Guide*. Addison-Wesley.
- COCKBURN, A. *Writing Effective Use Cases*. Addison-Wesley, 2001.
- FOWLER, M. *UML Distilled: A Brief Guide to the Standard Object Modeling Language*. Addison-Wesley.
- PRESSMAN, R. S.; MAXIM, B. R. *Software Engineering: A Practitioner's Approach*. McGraw-Hill.
- SOMMERVILLE, I. *Software Engineering*. Pearson.
- SCHWABER, K.; SUTHERLAND, J. *The Scrum Guide*. 2020.
- ISO/IEC/IEEE 29148. *Systems and software engineering — Life cycle processes — Requirements engineering*.



REGRAS



REQUISITOS



CASOS DE USO



COMPORTAMENTO



BACKLOG



SPRINT



TAREFAS



SOFTWARE

Modelar não é desenhar.

Modelar é tornar explícito aquilo que precisamos compreender antes, durante e depois de construir o software.